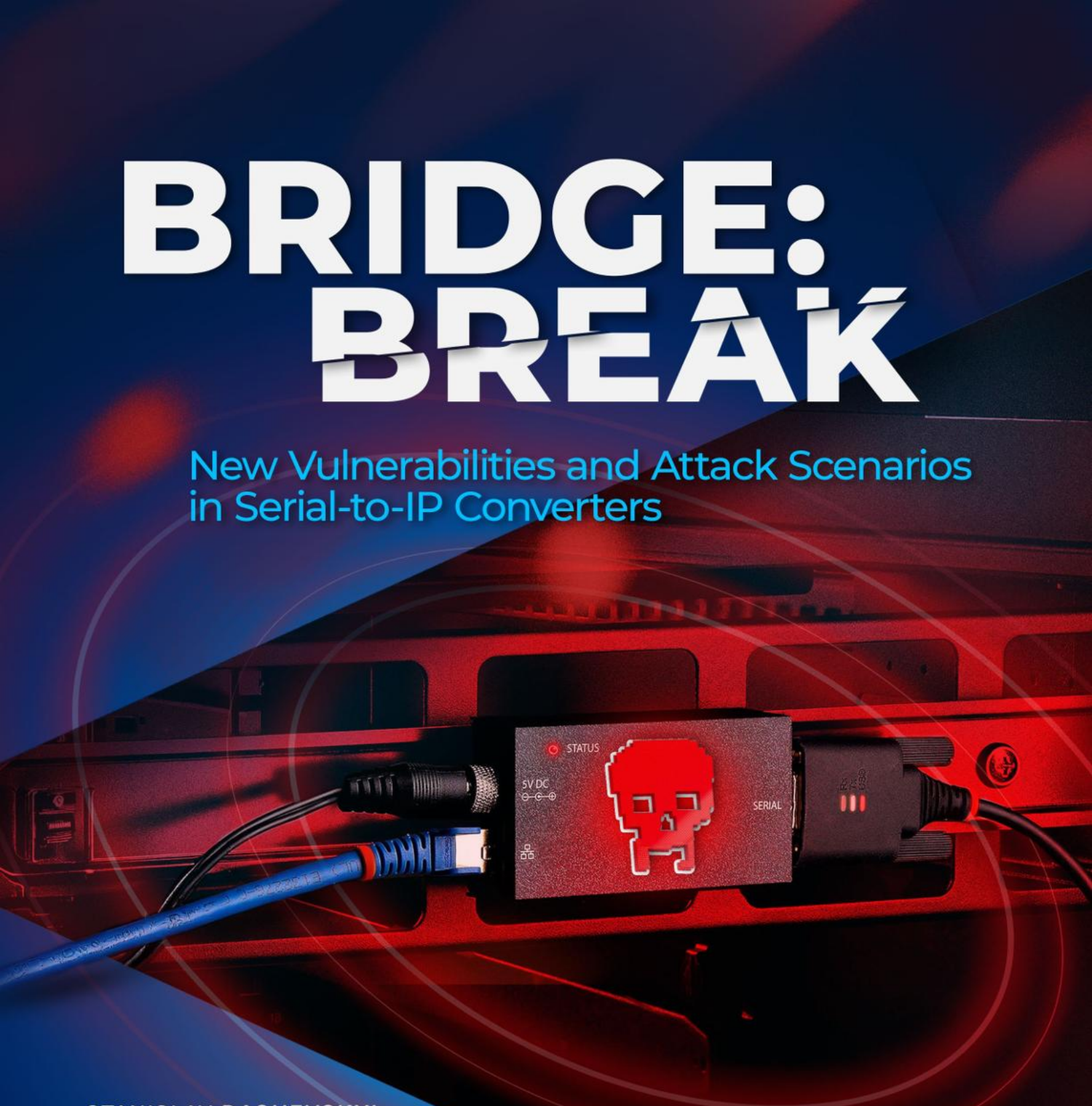
BRIDGE: BREAK

New Vulnerabilities and Attack Scenarios in Serial-to-IP Converters



STANISLAV **DASHEVSKYI**
FRANCESCO **LA SPINA**
DANIEL **DOS SANTOS**

FORESCOUT RESEARCH | VEDERE LABS

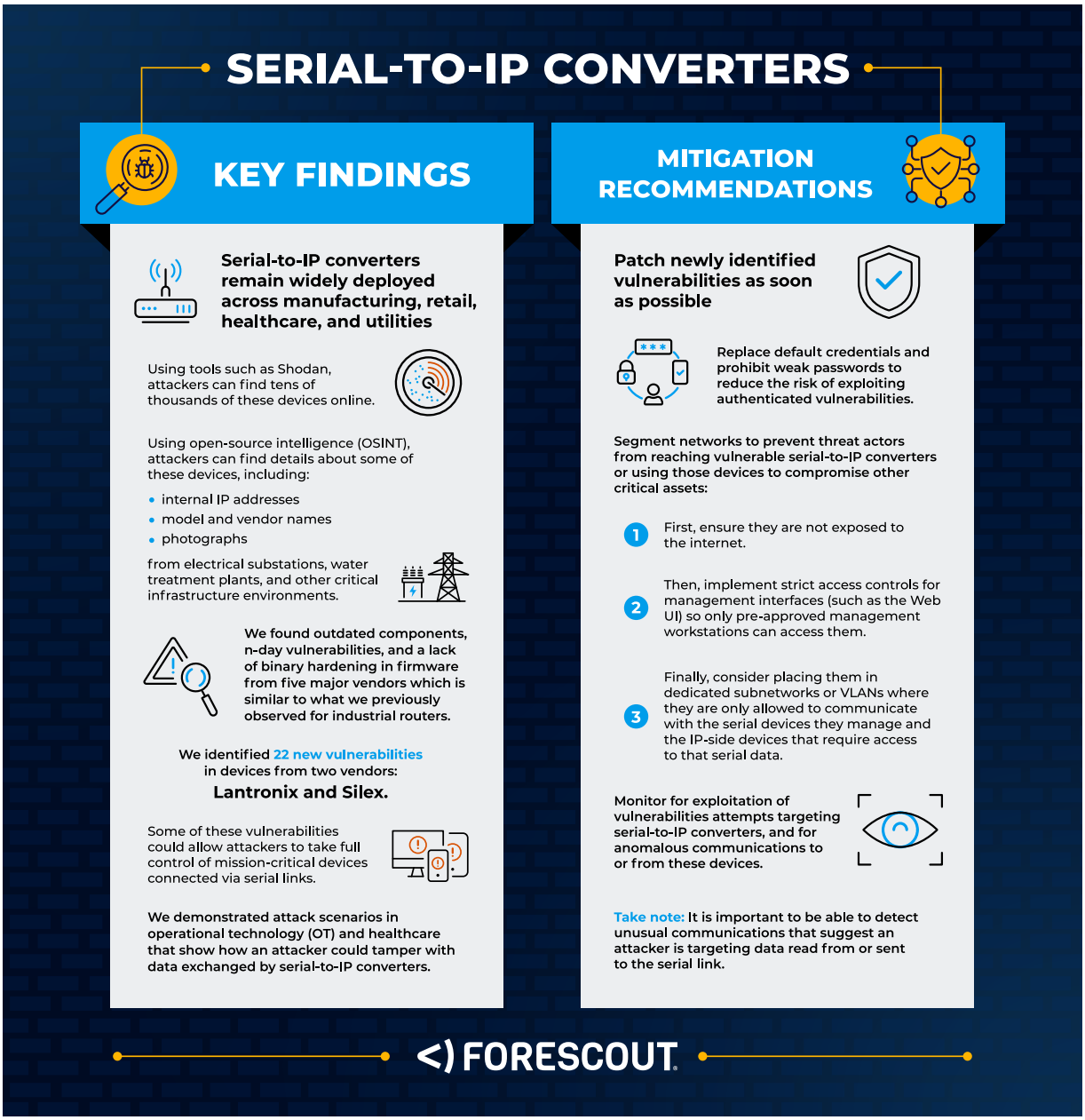
Contents

1. Executive Summary

Serial-to-IP converters may sound like ‘boring’ equipment whose only purpose is translating serial data into TCP/IP. In practice, they are necessary and widely deployed. They enable connectivity for medical devices in hospitals, programmable logic controllers (PLCs), sensors, and actuators in factories, and remote terminal units (RTUs), intelligent electronic devices (IEDs), and relays in electrical substations. Legacy serial equipment is not going away any time soon.

These seemingly innocuous ‘bridge’ devices can have outsized impact in critical environments. Threat actors do not find them boring at all. In 2015, an attack against Ukraine intentionally corrupted the firmware of several serial-to-IP converters, rendering electrical substations inoperable remotely. In 2025, these devices were again attacked in the Polish power grid.

As attacks on global critical infrastructure have become more common over the past decade, this report revisits the security of serial-to-IP converters.



2. Why Analyze Serial-to-IP Converters?

Serial-to-IP converters – also known as 'serial device servers' or 'serial-to-Ethernet adapters' – allow traditional serial equipment to communicate over modern IP networks. They translate serial data carried over RS-232, RS-422, or RS-485 into TCP/IP packets, and vice versa. This enables equipment designed for direct serial connections to be accessed remotely for monitoring and management. Figure 1 shows how a converter can connect many serial device types to local or remote IP networks.

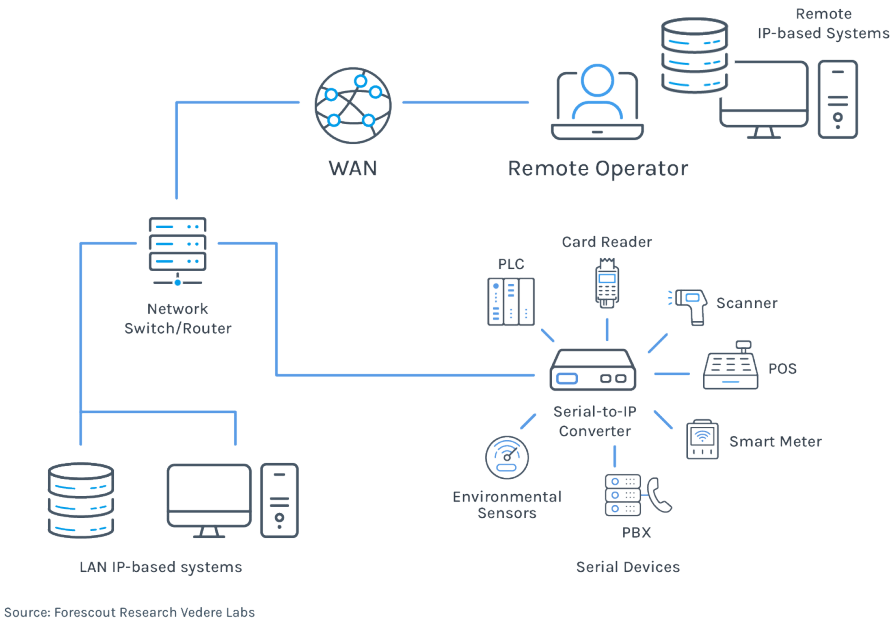


Figure 1 – Serial-to-IP converters

Serial-to-IP converters can connect to local switches using wired or wireless links. In some cases, they also support long-range wireless connectivity — similar to the OT/IoT cellular gateways and routers we investigated in previous research.

Devices that rely on serial link communications are widespread and are not going away any time soon. Market studies point to continued growth in the deployment of serial-to-IP converters over the next decade driven by the need to connect legacy equipment to modern networks.

Examples of serial equipment include: RTUs, relays and environmental sensors in the power grid, PLCs and computer numerical control (CNC) machines in industrial processes, barcode scanners and point-of-sale systems in retail, and bedside patient monitors in healthcare. Serial-to-IP converters allow organizations to onboard these devices into TCP/IP networks without replacing existing equipment.

2.1. Where They Are Used

Serial-to-IP converters are present across multiple critical infrastructure sectors. In 2024, market study data indicates these devices were used primarily in industrial applications, telecommunications, retail, healthcare, and utilities (see Figure 2).

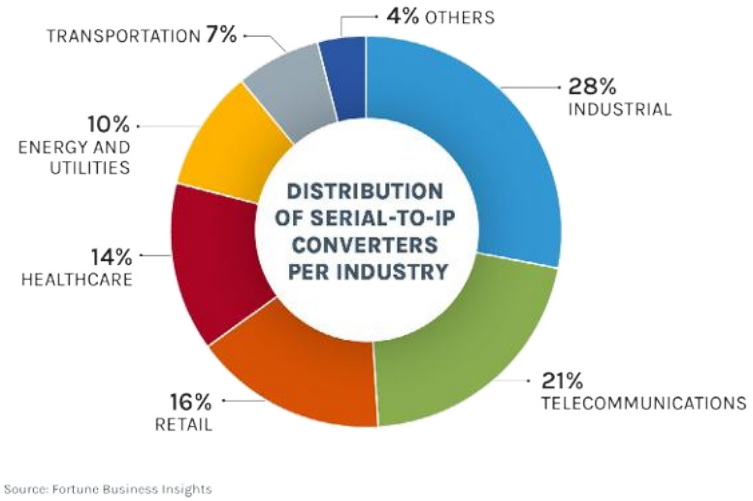


Figure 2 – Distribution of serial-to-IP converters per industry

Below, we examine use cases in two sectors – utilities and healthcare – in more detail, and point to references covering additional sectors.

Utilities: Electrical Substations and Water Treatment
Electrical substations transform electricity in the power grid from high to low voltage, or vice versa. They may be owned by utilities or large industrial customers.

Beyond the transformers, substations include a wide array of equipment used for monitoring and protection. For example, protection relays monitor variables, such as voltage, current, and frequency — and can trip circuit breakers when they detect dangerous conditions.

Many protection relays and other IEDs use serial communications and have been integrated into IP networks for remote monitoring and control via SCADA systems using serial-to-IP converters. A detailed description of how protection relays and other IEDs are connected in these networks, including how attackers may disrupt them is available on Mandiant's blog.

Figure 3 shows a simplified view of serial-to-IP converters in this architecture. While the figure shows a converter only on the substation side – with the control center SCADA operating over TCP/IP – some environments also use serial-to-IP converters in the control center to convert IP data back to serial for legacy applications.

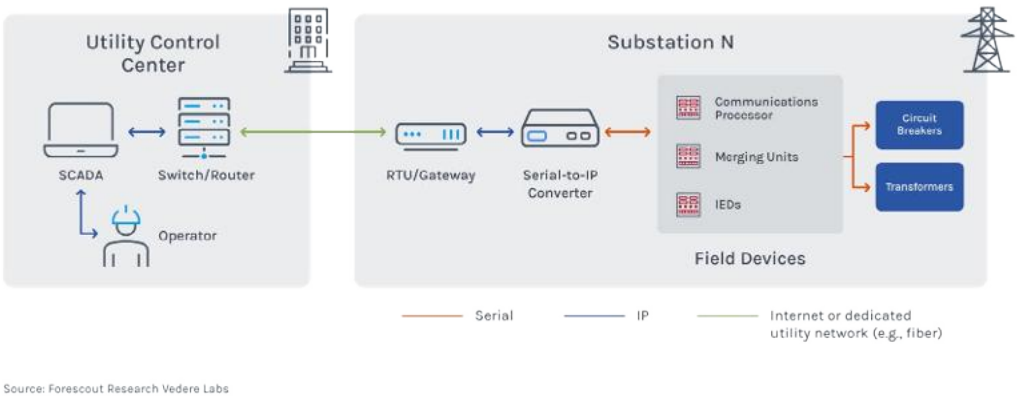


Figure 3 – Serial-to-IP converters in substations

Attackers who control serial-to-IP converter deployments can disrupt communications between remote operators and field devices (as previous incidents have shown; see Section 2.2) or inject malicious serial data to influence field device behavior.

Several detailed descriptions of serial-to-IP converter deployments in real electrical utility networks – sometimes including specific vendor and model information, and photographs of assets – are available via OSINT research, often in tender documents.

Examples include:

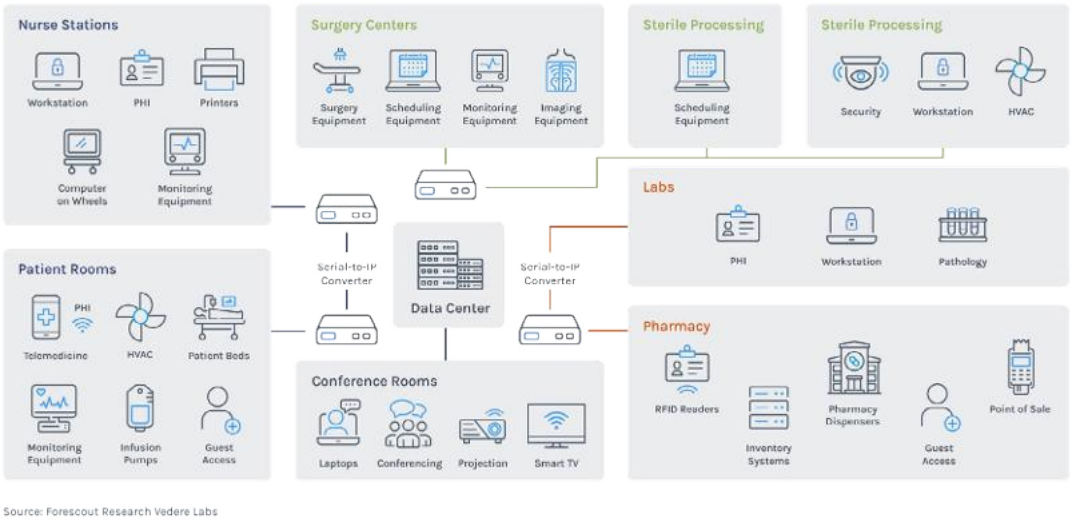
- The electric division of a city in Arizona where small single-port serial-to-IP converters were connected directly to existing serial communications processors to enable remote management of protection relays.
- The power utility of a city in Ohio where Lantronix EDS16PR serial-to-IP converters were used to connect the control center SCADA to RTUs and IEDs in substations via fiber optics.
- The national administration of power plants in a country in South America where IEDs, merging units, and intelligent terminals were connected to servers and workstations via Kyland SICOM* serial-to-IP converters.
- A transmission network in Australia where Lantronix serial-to-IP converters were used for remote monitoring and management of IEDs. The available report, published in 2015, notes that the Lantronix devices were already end-of-support with a replacement project underway.

A similar architecture is used in other types of utilities. In water and wastewater management, common use cases include: connecting PLCs and RTUs in water treatment facilities, pipelines, remote pump stations, and dams for remote monitoring and control as well as fault diagnosis. If the IEDs in Figure 3 are replaced with PLCs, and circuit breakers and transformers are replaced with valves and pumps, the simplified architecture remains comparable.

Similar levels of deployment detail are also available via OSINT for these utility environments. For example, a tender document from a water agency in Spain describes the use of Lantronix EDS32PR converters to connect to several GSM modems, including internal IP addresses for the devices and phone numbers for the modems. These modems transfer collected data from water facilities to a remote SCADA system.

Healthcare

Healthcare Delivery Organizations (HDOs), such as hospitals and clinics, are complex environments that include modern hardware and software alongside legacy equipment designed to last for decades. Figure 4 shows an example of these environments and where serial-to-IP converters may sit in their networks.



Source: Forescout Research Vedere Labs

Figure 4 – HDO network with serial-to-IP converters

The data center sits in the middle of the diagram and may contain Electronic Medical Record (EMR) systems, Personal Health Information (PHI), and billing and payment information. HDO networks also include connected medical devices in patient rooms, nurse stations, surgery centers, pharmacies, labs, and countless other locations. These devices may support clinical care or gather and monitor patient information to alert and inform clinical staff.

Examples of serial-connected instruments in this scenario include: blood chemistry analyzers, patient monitors, surgical lighting controllers, and environmental sensors in sterile compounding rooms. These devices may be connected to IP networks for remote monitoring of patient vital signs, to collect data for EMR systems, or for remote control of lighting.

HDO networks also include many non-clinical systems that may be connected via serial-to-IP converters, including cafeteria point-of-sale systems, vending machines, physical security systems, and smart building systems for energy management and HVAC.

On the positive side, these devices – clinical or not – typically do not communicate over wide area networks in the same way as the utilities example. However, hospitals have many more opportunities for untrusted parties to obtain physical access to devices and networks. In prior research, we discussed how HDO networks are often flat and lack proper segmentation, and how some segments may mix IoT devices, such as printers and time clocks, with medical equipment, including ultrasound systems connected via serial-to-IP converters.

Other Use Cases

Serial-to-IP converters support many other use cases that we do not explore in detail here. Examples include connecting:

- CNC machines for centralized data collection and operations monitoring in manufacturing plants.
- Field signaling controllers used in railway signaling for remote operation.
- Critical shipboard systems for maintenance, monitoring, and control, including propulsion and steering systems; and the Electronic Chart Display and Information System (ECDIS) used for navigation.
- Network equipment in data centers for out-of-band management.
- Fire alarm systems in building management networks.
- Gas pumps and automatic tank gauges (ATGs) for remote monitoring of tank levels in gas stations.
- Cash automation systems in financial institutions.
- Point of Sale (POS) terminals, barcode scanners, electronic scales, printers, and payment terminals in retail to improve synchronization of sales, inventory and membership data.

Estimating Serial-to-IP Converters prevalence

The total number of serial-to-IP converters in use worldwide is difficult to know, but our best estimate based on available market data and vendor reporting is in the millions. Because these devices are not intended to be exposed online, results from device search engines, such as Shodan, are incomplete. Table 1 shows close to 20,000 converters from major vendors observed online using simple queries. These queries may include false positives, and they likely underestimate the number of converters deployed globally. For context, Lantronix reported more than two million devices installed globally in 2012, and Moxa has stated that more than 82 million devices are connected. If one assumes an average of eight connected devices per converter, **it implies more than 10 million converters**.

Table 1 – Examples of serial-to-IP converters observed online

Vendor	Shodan Results	Top Countries
Lantronix	8,292	United States, 3038 Japan, 1584 Sweden, 780
Moxa	3,859	Russia, 1593 Taiwan, 350 United States, 325
Digi OpenGear	2,946	United States, 1800 Australia, 178 United Kingdom, 136
Digi RealPort	2,985	Italy, 662 United States, 493 Spain, 383
Digi PortServer	515	Japan, 378 United States, 55 Mexico, 44
Perle	241	United States, 121 Germany, 50 Canada, 32

2.2. Previous Incidents and Research

Individual vulnerabilities in serial-to-IP converters have been reported for more than 20 years. The earliest example we identified was CVE-2005-2189 which was an authentication bypass in Lantronix SecureLinx devices that allowed attackers to retrieve private Secure Shell (SSH) keys. Since then, hundreds of similar issues have been reported across widely deployed products from vendors, such as Lantronix, Moxa, Digi, and others.

The earliest security research we found that examined serial-to-IP products beyond individual vulnerabilities was H.D. Moore's 2013 presentation "Serial Offenders". It focused on Digi and Lantronix devices, described more than 200,000 exposed assets, discussed insecure configurations (including default credentials and direct access to serial data), and highlighted critical infrastructure use cases.

Because serial-connected assets control critical functions, and serial-to-IP converters sit at the intersection of serial and TCP/IP networks, threat actors have also exploited this communications layer in real incidents:

- **2015**: Several Ukrainian power distribution companies were impacted by a Russian cyberattack. Among other actions, the attackers loaded corrupted firmware onto Moxa serial-to-IP converters via the firmware update function. This rendered the converters inoperable and cut communications between control centers and substations. Operators could no longer remotely close circuit breakers and had to resort to manual operations, delaying recovery from the outage.
- **2016**: A follow-on attack on Ukraine's power grid used the Industroyer malware. Its IEC-101 payload communicated with field RTUs via serial links exposed through Windows COM ports. The malware used one port for malicious communication and opened the remaining COM ports to prevent legitimate access from using them.
- **Early 2022**: Following Russia's full-scale invasion of Ukraine, several hacktivist groups targeted operational technology in both countries. In this activity, GhostSec shared a list of Moxa OnCell cellular gateways – which can also connect serial devices – used in Russia, and posted screenshots of defaced web interfaces.

- **Late 2022**: A Russian attack against Ukraine's power grid targeted Hitachi's MicroSCADA supervisory control systems using the SCIL programming language. Although unconfirmed, some issued SCIL commands may have targeted RTUs via serial IEC-101 communications, similar to Industroyer's original approach.
- **2025**: A Russian attack on Poland's Grid Connection Points (GCPs) – substations dedicated to distributed energy resources – targeted Moxa NPort serial device servers. The attackers reportedly logged in using default credentials, restored the devices to factory settings, changed login passwords and set IP addresses to unreachable values. The intent was to cut communications and delay recovery.

In the years following the first two incidents above, additional security research examined serial-to-IP converters and adjacent use cases:

- **2015**: Dennis Maldonado presented at DEF CON on how to compromise physical access control systems via their serial ports by leveraging serial-to-IP converters.
- **2016**: K. Reid Wightman presented an analysis of Moxa and Digi products at S4, including new vulnerabilities and vendor responses. Shortly afterward, Joakim Kennedy disclosed CVE-2016-1529 affecting Moxa products and demonstrated an attack on a serial glucose meter.
- **2017**: Ankit Anubhav reported that thousands of Lantronix devices were leaking passwords online due to a vulnerability known since 2012.
- **2018**: Ken Munro demonstrated how serial-to-IP converters could be used to compromise shipboard networks.
- **2020**: Jason Larsen presented at S4 a security assessment of an electric system that included pushing malicious firmware to Moxa devices.

Since then, we found limited new research beyond additional single vulnerability disclosures in widely deployed serial-to-IP products. Our deep lateral movement research from 2023 showed how serial links can be abused to move between Purdue Model Level 1 assets. This report revisits the topic by assessing the security of current products from popular manufacturers.

3. Main Findings

Major manufacturers of serial-to-IP converters include Moxa, Digi, Advantech, Lantronix, Perle, Silex, and others. To assess the current security posture of this product category, we analyzed serial-to-IP converters from leading manufacturers whose firmware we could obtain and unpack. Table 2 summarizes the selected products.
To keep the focus on cross-vendor security patterns (rather than specific vendors) we do not name the specific vendors and models in Table 2. We identify vendors and models only where we discovered new vulnerabilities and coordinated disclosure. For each product, we analyzed the latest firmware version available at the time of our analysis.

Table 2 – Selected devices

Device	Linux kernel version	OS/Build tool
A	v4.4.32	Buildroot 2017.02
B	v6.12.22	Linux-based OS
C	v3.6.5	Buildroot 2014.04
D	v5.10.140	OpenWRT
E	v3.14.0	Linux-based OS
F	v4.1.15	Buildroot 2017.02

We conducted the research in two phases:

- Automated analysis of software composition, historical vulnerabilities and binary hardening across the firmware listed in Table 2
- Manual analysis to identify new vulnerabilities in a subset of the products

3.1. Software Composition

We used the open-source EMBA tool to identify open-source software components in each firmware image, and to match identified versions to potential vulnerabilities and evaluate binary hardening. **We report component versions, the number of vulnerabilities, publicly available exploits, and binary hardening indicators as generated by EMBA. These results may be incomplete or partially incorrect due to false positive and false negative matches, but they provide a useful high-level view of the security posture across the analyzed devices.**

The results, detailed below, are broadly comparable to our previous research on OT routers:

- The open-source software stacks in serial-to-IP converters resemble those we observed in routers, even though the products serve different functions. Examples include operating systems based on OpenWRT and Buildroot, and common libraries, such as OpenSSL.
- The age of Linux kernels—and by extension other software components—correlates with the number of known vulnerabilities and potential exploits.
- Binary hardening is inconsistent — with some vendors applying only a subset of available techniques.

Vendors often prioritize stability over security which can lead to longer software update cycles. When combined with incomplete binary protections, this can create environments that are easier to exploit. **For example, Section 3.2 discusses how a vulnerability in the net-snmp version used by Silex—and known for more than 10 years—can be triggered by a malformed packet to crash the Simple Network Management Protocol (SNMP) service.**

Software Versions

Table 3 shows a selection of software components that were common across the firmware images we analyzed. Several points stand out:

- **On average, each firmware contained 80 identified components**, but the variation was significant. Firmware F had 17 identified components while firmware D had 248.
- **Vendors adopted very different Linux kernel branches**. Half were end-of-life (EOL) which complicates future updates. The other half adopted kernel branches expected to be supported for longer periods which is encouraging:
  - The earliest branch in Table 3, 3.6, reached EOL in December 2012.
  - The 3.14 long-term support (LTS) branch reached EOL in August 2016.
  - The 4.1 LTS branch reached EOL in May 2018.
  - The 4.4 branch was selected as the first super-long-term support (SLTS) kernel in the Civil Infrastructure Platform (CIP) and is scheduled to reach EOL in January 2027.
  - The 5.10 CIP kernel is also SLTS and will be maintained until January 2031.
  - CIP's 6.12 SLTS kernel support is set to run until mid-2035
- **Kernel selection tends to 'drag' other component versions** which can indicate slower update cycles:
  - For example, firmware B has the newest kernel and one of the most recent OpenSSL releases.
  - Firmware E has the oldest kernel and the oldest library versions in the group.
  - The same pattern appears for zlib, curl, dnsmasq and other components.
- **Aside from the Linux kernel, only three components were identified in every firmware image**: OpenSSL, sed, and zlib:
  - OpenSSL is a foundational cryptography library. Most vendors use versions 1.0.2 and 1.1.1. Security fixes for these versions are only available via dedicated enterprise support.
  - Firmware B uses OpenSSL 3.4 which has regular support available until October 2026.

Table 3 – Identified components

Component	A	B	C	D	E	F	Latest (stable) releases
Total identified components	34	90	64	248	31	17	N/A
Linux Kernel	4.4.32	6.12.22	3.6.5	5.10.140	3.14.0	4.1.15	3.6.11 (EOL) 3.14.79 (EOL) 4.1.52 (EOL) 4.4.302 (SLTS) 5.10.250 (SLTS) 6.12.68 (SLTS)
openssl	1.1.1w	3.4.0	1.1.1d	1.1.1q	1.0.2g	1.0.2k	3.0 onwards
sed	4.0	4.0	4.0	4.0	4.0	4.0	4.9
zlib	1.2.11	1.2.8 | 1.3.1	1.2.13 | 1.2.3	1.2.11-6	1.1.4	1.2.11	1.3.1
busybox	1.26.2		1.34.1	1.35.0-11 | 1.35.0-3	1.22.1 | 1.4	1.26.2	1.36.1
ethtool	4.8	6.7	4.0	5.16-1		4.8	6.15
net-snmp	5.9.1	5.9.3	5.8	5.8		5.7.3	5.9.4
curl		8.9.1	7.61.0	7.66.0-1	7.49.1		8.17.0
dnsmasq		2.90	2.83	2.86-15	2.79		2.91
iptables	1.6.1	1.8.10	1.4.21			1.6.1	1.8.11
mtd-utils	1.5.2	2.0.2	1.5.1	2.1.4			2.3.0
ncurses		6.5.20240427	5.9.20110404	6.3.20211021	5.9.20110404		6.5
udhcp	1.26.2		1.34.1	1.35.0		1.26.2	1.36.1 (Busybox)
util-linux		2.37.2	2.26	2.37.3	2.21		2.41.2
wpa_supplicant	2.9	2.11			2.6	N/A	2.11

Historical Vulnerabilities

Figure 5 shows the number of previously known vulnerabilities affecting the kernel and other software components in each firmware image, based on EMBA's matching:

- **On average, each firmware image had**:
  - **2,255 known vulnerabilities affecting the kernel**.
  - **212 affecting other open-source components**.
  - The kernel had the highest number of vulnerabilities in every case except B.
- **Kernel age and the number of kernel CVEs are strongly correlated**. This is expected: as time passes, more vulnerabilities are discovered and no single kernel branch stands out as vulnerable relative to its age.
- Kernel age does not directly correlate with the number of non-kernel component vulnerabilities because that depends on the number of identified components. However, **kernel age does correlate with the average number of vulnerabilities per component** (shown on the right-hand side of the figure):
  - Firmware B had the lowest total number of vulnerabilities (210), followed by D (2404) and F (2808).
  - However, the lowest average vulnerabilities per component was in D.

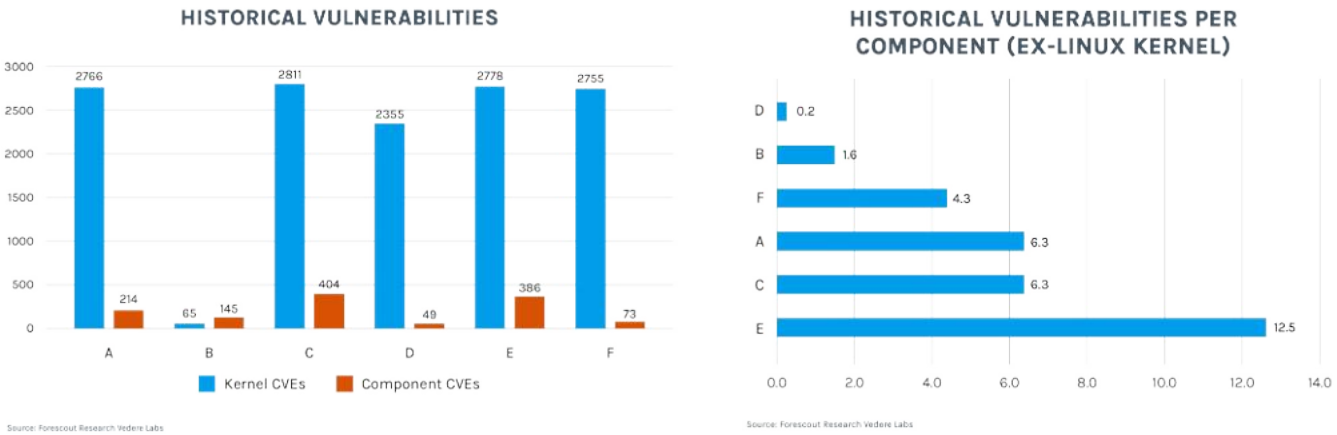


Figure 5 – Number of CVEs affecting each firmware

Not every vulnerability is equally severe, so we also examined severity and exploit availability. Figure 6 breaks down historical vulnerabilities by CVSSv3 score and the number of publicly available exploits:

- On average, firmware images had 1,566 vulnerabilities with low or medium severity (68%), 675 with high severity (29%), and 63 with critical severity (3%).
- On average, firmware images had 89 publicly available exploits.

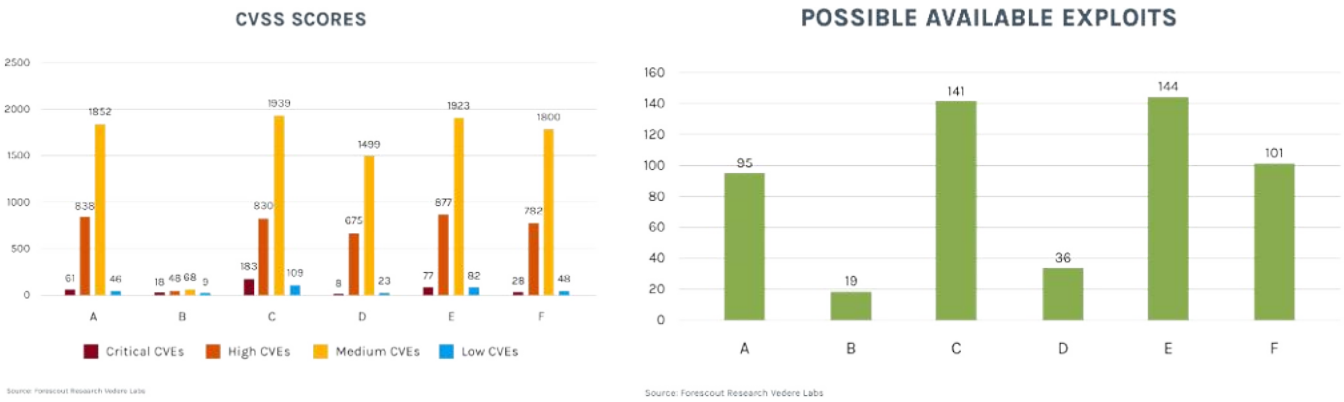


Figure 6 – Number of CVEs by CVSSv3 score and publicly available exploits

Binary Hardening

To assess binary hardening practices in each firmware, we considered the following features:

- **RELRO** (Relocation Read-Only): This feature ensures that the linker resolves all dynamically-linked functions at the beginning of the program execution and sets the Global Offset Table (GOT), which are used for resolving function addresses in shared libraries, to read-only. This prevents attackers from arbitrarily redirecting execution by overwriting the GOT.
- **PIE** (Position Independent Executable): This mechanism loads all functions of the target binary and its dependencies into arbitrary locations in virtual memory making it much more difficult to launch Return-Oriented Programming (ROP) attacks.
- **Stack canaries**: These are secret, randomly-generated values placed onto the stack of a function. Before a function returns, the canary value is checked. If it has been altered, the program exits. This prevents straightforward stack smashing exploits, although it does not prevent scenarios when stack canaries may be 'leaked' to the attacker.
- **NX** (No-eXecute bit): This hardware feature allows an operating system to mark certain memory pages as non-executable preventing code from running in these areas. When used, the stack and heap areas are marked as non-executable which prevents straightforward buffer overflow exploits.

- **Symbols in binaries**: Symbols are used for debugging purposes but also aid attackers by simplifying reverse engineering and exploitation. Since most binaries in question are open source, the presence of debugging symbols does not give a significant advantage.

Figure 7 shows the percentage of all binaries, including executables, shared libraries, and kernel modules, within each analyzed firmware image that have specific hardening features present:

- On average, binaries across firmware images used the following protections:
  - 23% used stack canaries. This was the least common protection overall and only B made significant use of it.
  - 44% used RELRO. This average is skewed because B, C and D applied this protection broadly, while others did not.
  - 84% used NX. This was the most common protection, which helps prevent a class of trivial buffer overflow exploits.
  - 67% used PIE. Only B had close to 100% of binaries with PIE enabled. Wider adoption of PIE would be preferable, particularly given the limited use of stack canaries. Address space layout randomization (ASLR) is significantly less effective when PIE is not used, and the absence of stack canaries further lowers the bar for implementing buffer overflow exploits.
- Overall, vendors with newer kernels —and, consequently, fewer known vulnerabilities—also tend to apply binary protections more consistently.

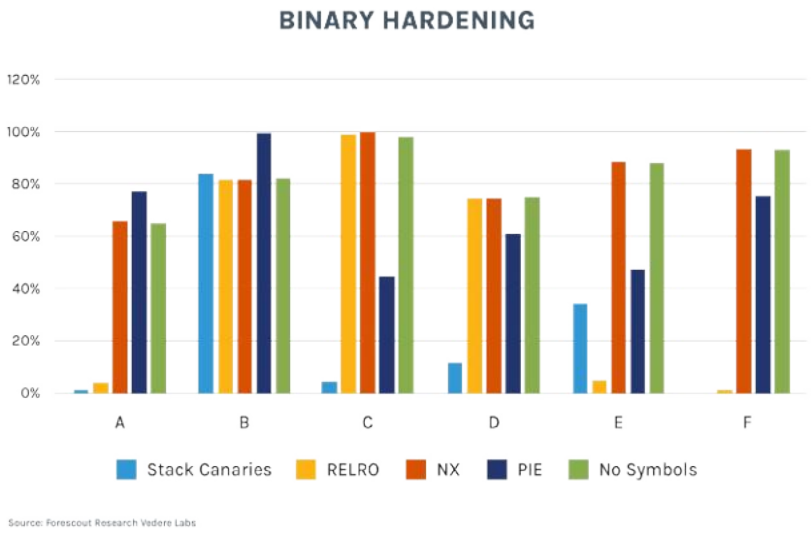


Figure 7 – Binary hardening across firmware images

3.2. New Vulnerabilities

Due to time constraints, we focused our manual analysis on the following products and firmware versions:

- Lantronix EDS3000PS Series: Compact, office-grade multi-serial-port serial device server (8–16 ports). Firmware analyzed: 2.1.0.0R3.
- Lantronix EDS5000 Series: Rack-mountable serial device server supporting 8, 16, or 32 serial ports. Firmware analyzed: 3.1.0.0R2.
- Silex SD330-AC: Small serial device server designed to connect RS-232C serial devices over a wireless network or an Ethernet port. Firmware analyzed: 1.42.

We found eight new vulnerabilities affecting Lantronix products and 12 affecting Silex products. We also confirmed that two n-days affected Silex products: CVE-2015-5621 and CVE-2024-24487. Table 4 through Table 6 list the vulnerabilities. We discuss selected vulnerabilities in more detail in Part 2 (“Technical Deep-dive”). At a

high level, the findings fall into the following impact categories:

- Remote code execution (RCE) via OS command injection or memory corruption (for example, buffer overflows).
- Device takeover via authentication weaknesses.
- Firmware tampering enabled by a hardcoded signing key.
- Denial of service (DoS).
- Other impacts, including arbitrary file upload, authentication bypass, and information disclosure (including passwords and keys) due to weak encryption.

Table 4 – New vulnerabilities affecting Silex SD-330AC

Vuln ID	Description	Potential Impact
CVE-2026-32955	Stack overflow in login redirection URL query parameter (password required)	RCE
CVE-2026-32956	Heap overflow in redirection URL query parameter (authentication not required)	RCE
FSCT-2025-0021	Password change via “console socket” (/tmp/spshttpio) or the Web UI does not prompt for the old password. Silex did not assign a CVE ID to this issue, stating: “Although this is not a desirable specification from a security standpoint, since the password cannot be changed without logging into the device, this item alone is not considered a vulnerability.” We still consider this behavior relevant because it could be chained with other vulnerabilities to enable privilege escalation.	Device takeover
CVE-2026-32957	Unauthenticated attackers can upload firmware binaries via a POST request	Arbitrary files upload
CVE-2026-32958	Firmware-signing key may be obtainable by attackers. **Silex is remediating this issue; details will be available later**	Firmware tampering
CVE-2026-32959	Data between AMC Manager and Silex products is transferred via a proprietary protocol (sx_smpd) in clear text or using weak encryption, enabling man-in-the-middle attacks, and disclosure of sensitive data (for example, passwords and configurations). Also affects Silex AMC Manager and potentially other products.	Information disclosure
CVE-2026-32960	Failure to clear TCP data buffers for sx_smpd can allow credential reuse via spoofed authentication.	Auth. Bypass
CVE-2026-32961	sx_smpd UDP packet data length is not validated, which can lead to heap-based buffer overflows and other memory corruption.	DoS/RCE
CVE-2026-32962	Device settings (for example, WiFi and Ethernet) can be configured through the “jcpd” management protocol (“Serial Device Server Setup”) without authentication, enabling configuration tampering that can lead to DoS, information disclosure (via manipulated network communications), and other impacts.	Configuration tampering
CVE-2026-32963	Unauthenticated reflected cross-site scripting on the System Status page.	Client-side code execution
CVE-2026-32964	jcpd (“Serial Device Server Setup”) configuration arguments do not filter certain special characters (for example newlines), enabling unauthenticated injection of arbitrary entries into system configuration files.	Configuration tampering
CVE-2026-32965	If the admin password is not set (for example, after a factory reset), anyone on the local network can configure the device via sx_smpd by authenticating with an empty (NULL) password.	Device takeover
CVE-2015-5621	net-snmp daemon (snmpd) affected by CVE-2015-5621: a malformed UDP packet can crash snmpd, which is not automatically restarted; reboot required to restore service.	DoS
CVE-2024-24487	Unauthenticated remote attackers can trigger DoS via crafted UDP packets using the EXEC REBOOT SYSTEM command of jcpd (“Serial Device Server Setup”).	DoS

Table 5 – New vulnerabilities affecting Lantronix EDS3000PS

Vuln ID	Description	Potential Impact
CVE-2025-67039	Authentication on management pages can be bypassed by appending a specific suffix to the URL and sending an Authorization header with "admin" as the username.	Auth. bypass
CVE-2025-67041	The host parameter of the TFTP client in the Filesystem Browser page is not properly sanitized, allowing command injection that leads to arbitrary command execution as root.	RCE
CVE-2025-70082	Password change via the Web UI does not prompt for the old password which could allow an attacker to become an administrator, disrupt serial communications, and lock out legitimate administrators.	Device takeover

Table 6 – New vulnerabilities affecting Lantronix EDS5000

Vuln ID	Description	Potential Impact
CVE-2025-67034	Authenticated attackers can inject OS commands into the "name" parameter when deleting SSL credentials via the management interface; commands execute as root.	RCE
CVE-2025-67038	HTTP RPC module writes logs via a shell command on authentication failure; the username is concatenated without any sanitization, allowing OS command injection; commands execute as root.	RCE
CVE-2025-67036	Log Info page allows log files viewing by filename; missing sanitization in the filename parameter enables OS command injection; commands execute as root.	RCE
CVE-2025-67035	SSH Client and SSH Server pages are affected by multiple OS injection issues due to missing input sanitization in delete actions (for example, server keys, users, and known hosts); commands execute as root.	RCE
CVE-2025-67037	Authenticated attackers can inject OS commands into the "tunnel" parameter when killing a tunnel connection; commands execute as root.	RCE

4. Attack Scenarios

Attackers can achieve at least three types of impact when targeting serial-to-IP converters:

- **Denial of service**: Prior incidents in 2015 and 2025 illustrate how adversaries can disrupt serial communications with field devices, even using different methods (firmware corruption in the first case and setting unreachable IP address values in the second). In this report, we show how firmware vulnerabilities can enable time-triggered denial-of-service conditions that attackers could synchronize with other actions.
- **Lateral movement**: in our previous deep lateral movement research, we demonstrated that attackers could cross boundaries of non-routable OT networks by exploiting controller-level devices. Here, we focus on exploiting serial
- **Sensor and actuator data tampering**: Attackers may alter serial data received from a sensor as it moves into the IP network. For example, changing temperature, pressure, humidity, flow, patient heart rate readings to arbitrary values. Conversely, attackers may modify commands traveling from the IP network to the serial side before they reach an actuator. For example, changing the speed or direction of a servo motor. One well-known example of this type of operator deception is Stuxnet which manipulated what operators saw while malicious changes were made in the underlying process. Although Stuxnet operated via PLCs, similar 'manipulation of view' outcomes could be achievable in some environments through exploitation of a serial-to-IP converter. This aligns with the general concept captured in MITRE ATT&CK for ICS technique T0832 – Manipulation of View where adversaries manipulate what operators see to influence decisions and outcomes.

Once a converter is compromised:

- Attackers can tamper with serial communication in both directions.
- Serial protocols often lack authentication or encryption.
- As a result, attackers may be able to influence devices and processes that rely on that serial link.

Serial-to-IP converters are typically not intended to be exposed to the internet, but attackers may still reach them through several initial access vectors. In Poland, initial access was reportedly achieved via internet-facing VPN (virtual private network) concentrators. In other environments, access could be gained through compromised IT workstations or via deep lateral movement in OT networks. In hospitals, access could also come from physical access to patient rooms and flat networks.

In the following sections, we assume an attacker can reach a vulnerable serial-to-IP converter. We then walk through concrete examples of how the new vulnerabilities in Section 3 could be exploited to cause operational impact. Scenario 1 covers data tampering in OT (examples include: a water treatment facility, a power substation, or a manufacturing plant) while Scenario 2 covers denial of service in healthcare.

4.1. Scenario 1: Data Tampering in OT

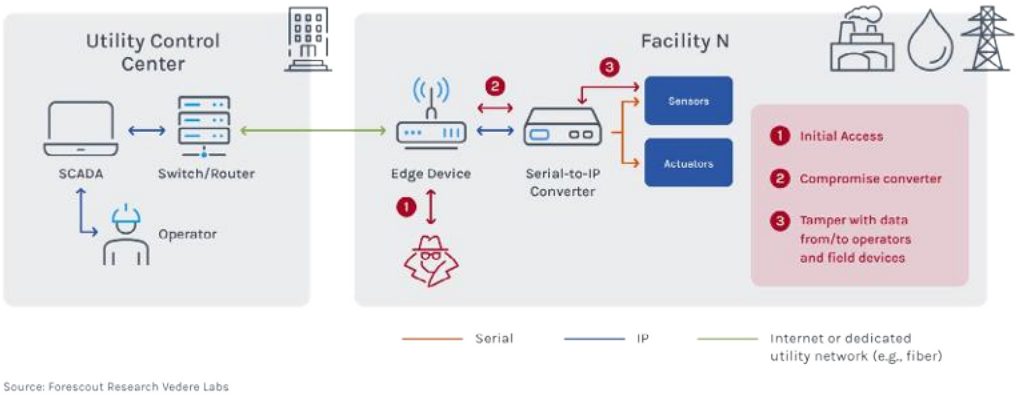


Figure 8 – Attack scenario on utilities

Figure 8 expands on the simplified architecture shown in Figure 3. It abstracts sector-specific details to make the scenario applicable across power, water, and manufacturing, and illustrates a representative attack flow:

1. **Initial access**: The attacker gains access to a remote facility through an Internet-exposed edge device, such as an industrial router, firewall, or VPN concentrator. This is consistent with reported initial access vectors in the Poland GCP incidents in 2025 or the initial access vector for substations in the Denmark attacks in 2022 (although the latter did not involve serial-to-IP converters). Other access paths in remote facilities can include compromised IP cameras used against the Indian power grid in 2022, other misconfigured IoT equipment, social engineering, or malware on local workstations.
2. **Converter compromise**: The attacker compromises the serial-to-IP converter through vulnerable remote management protocols (CVE-2026-32958, CVE-2026-32959, CVE-2026-32964), authentication weaknesses (CVE-2026-32960, CVE-2025-67038), or RCE vulnerabilities (CVE-2026-32955, CVE-2025-67038, CVE-2025-67035). In practice, the attacker may first perform discovery and lateral movement inside the facility network; we omit these steps here for clarity. A full description of other kinds of devices that would likely be found in that local network is given in Mandiant's blog for a substation.
3. **Data manipulation**: After achieving code execution on the converter, the attacker tampers with serial data traveling to or from the IP network. In our lab, we connected a serial thermometer to a simple SCADA application displaying the ambient temperature. Before the attack, the application showed a stable temperature around 24 °C. After the attack, the graph oscillated between -40 °C to +40 °C. Conversely, the attacker could also make an oscillating temperature appear stable.

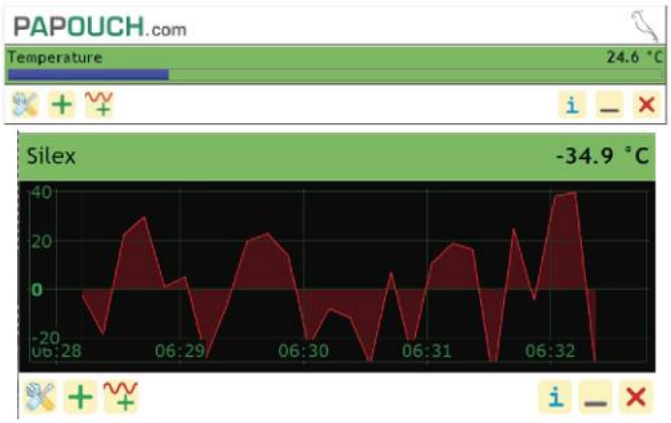


Figure 9 - Example attack on a serial thermometer connected to SCADA

Tampering with temperature readings can affect control systems across utilities, manufacturing, and healthcare. Attackers could also target readings and commands from more specialized sensors and actuators. Doing so may require familiarity with sector-specific protocols beyond plain text, such as IEC-60870-5-101 in utilities, Modbus RTU in manufacturing, or ASTM E1394 in healthcare. However, these protocols are not particularly complex, and the absence of encryption in many serial deployments can still enable tampering. Achieving significant physical impact may also require process-specific knowledge for complex environments, but manipulating sensor readings can be dangerous because it can conceal conditions that would otherwise trigger human intervention.

4.2. Scenario 2: Denial of Service in Healthcare

Over the past decade, cyberattacks have contributed to major financial losses and delays in patient care across healthcare organizations. In many cases, the operational impact has been driven by ransomware activity causing operational disruption and increases the pressure on affected organizations.

In this scenario, we show how attacks on serial-to-IP converters could become part of a ransomware campaign or state-sponsored operation that extends beyond data disruption to affect medical devices. In prior research, we discussed how such outcomes could be achieved by targeting multiple medical devices directly. That approach is harder to scale because it typically requires exploiting different vulnerabilities across many distinct device types. **Targeting serial-to-IP converters, by contrast, can interrupt communications for multiple connected devices at once, disrupting diagnostic workflows and care delivery.**

In this scenario, the threat actor would need to:

1. **Weaponize firmware**: Modify a firmware image by exploiting a vulnerability, such as CVE-2026-32958. The weaponized firmware could trigger its effects immediately after installation or embed time-triggered logic. Potential outcomes could include bricking the converter, stopping serial communications, wiping configurations, or attempting to spread to other similar converters by exploiting additional vulnerabilities.
2. **Deliver the firmware**: Push the weaponized firmware to target devices. This could occur after compromising the IT network and exploiting a vulnerability to upload firmware remotely (for example, CVE-2026-32957) or through a targeted phishing campaign that persuades biomedical engineers to install a malicious ‘security firmware update’.

Once activated, the weaponized firmware could cause serial-to-IP converters stop responding on the network. Potential impacts include:

- Analyzers stop reporting results to laboratory information systems, creating processing backlogs.
- Surgical lighting controllers become unresponsive to remote commands.
- Infusion pump calibration and certification workflows are halted.
- Telemetry from environmental sensors is interrupted.
- Patient monitors lose network connectivity.

Where feasible, clinical staff would need to revert to manual procedures. Recovery would likely require engineers to physically replace affected serial-to-IP converters. Given the number of deployed converters in many environments, replacement and revalidation can prolong disruption even after the initial incident has been contained.

5. Conclusion and Recommendations

This research highlights weaknesses in serial-to-IP converters and the risks they can introduce in critical environments. As these devices are increasingly deployed to connect legacy serial equipment to IP networks, vendors and end-users should treat their security implications as a core operational requirement.

Based on the new vulnerabilities and attack scenarios we demonstrated and supported by prior attack evidence and the availability of deployment information, we recommend patching vulnerable devices as soon as possible:

- Lantronix has released two firmware updates that address the issues: 2.2.0.0R1 for the EDS5000 series and 3.2.0.0R2 for the EDS3000 series.
- Silex has also released updates to address the issues.

In addition to patching, we recommend:

- Replace default credentials and prohibit weak passwords to reduce the risk of exploiting authenticated vulnerabilities.
- Segment networks to prevent threat actors from reaching vulnerable serial-to-IP converters or using those devices to compromise other critical assets:
  - First, ensure they are not exposed to the internet.
  - Then, implement strict access controls for management interfaces (such as the Web UI) so only preapproved management workstations can access them.
  - Finally, use dedicated subnetworks or VLANs where they are only allowed to communicate with the serial devices they manage and the IP-side devices that should have access to that serial data.
- Monitor for exploitation attempts on serial-to-IP converters and for unusual communication patterns that suggest an attacker is targeting data read from, or sent to, the serial link.

Recommendations to Vendors:

- Follow secure-by-design principles that treat security as a business requirement.
- Implement a secure development lifecycle (SDLC) that embeds security into each phase of software development.
- Use Linux kernel versions that are as recent as practical and are supported over a long lifecycle.
- Maintain an inventory of software components shipped in firmware and update those with known vulnerabilities.
- Implement binary hardening to make exploitation more difficult.
- Perform regular security testing of implemented protocols, software, devices, and platforms to identify issues before release. Focus on commonly targeted interfaces, such as web management UIs.
- Adopt well-vetted protocols and use signing and encryption algorithms.
- Always use asymmetric cryptography for firmware signing and signature verification.
- Consider identifying devices exposed online and notifying customers about this misconfiguration.

Part 2: Technical Deep-Dive

This section provides additional technical detail for selected vulnerabilities introduced in Section 3.

Silex Vulnerability Details

CVE-2026-32955: Authenticated stack overflow in login redirection URL
A stack overflow in the login redirection URL can allow an attacker who knows the administrator password to execute arbitrary code. An arbitrarily long string can be appended to the `page _ url` query parameter during the login POST request to `/ssi/public/login/login _ main.htm`.

Vulnerable code path (ssi binary):

```
1.  LABEL_25:
2.    v14 = dword_33750;
3.    if ( strchr((const char *)dword_33750, 63) )
4.      v15 = '&';
5.    else
6.      v15 = '?';
7.    sprintf(
8.      buffer,
9.      "%s%cpage_url=%s&page_id=%s&page_sub=%s&language=%d&access=%s",
10.     v14,
11.     v15,
12.     page_url,
13.     v12,
14.     v13,
15.     *dword_33528,
16.     byte_337C4);
17.   flock(v0, 8);
18.   close(v0);
19.   printf("Location:%s\r\n\r\n", buffer);
20.   return 302;
21. }
```

The `sprintf` at line 7 uses a buffer of 288 bytes, and copies `page _ url` into it without bounds checking. If `page _ url` exceeds the buffer size, this can trigger a buffer overflow. Exploitation requires knowledge of the administrator password.

CVE-2026-32956: Unauthenticated heap overflow in redirection handling
A heap overflow in the redirection URL handling might allow an unauthenticated attacker to execute arbitrary code.

An attacker can append an arbitrarily long string to a GET request for a protected page (for example, `/ssi/secure/pass _ main.htmAAAAA[..]`). The unauthenticated requester is redirected to the login page, and the originally requested path is copied into the `page _ url` query parameter.

Vulnerable code path (ssi binary, main function):

```
1.  LABEL_52:
2.      if ( *v27 == '/' )
3.        ++v27;
4.      v35 = dword_33A70;
5.      if ( strchr((const char *)dword_33A70, '?') )
6.        v36 = '&';
7.      else
8.        v36 = '?';
9.      sprintf(
10.       (char *)url_buffer,
11.       "%s%cpage_url=%s&page_id=%s&page_sub=%s&language=%d",
12.       _login_url,
13.       v36,
14.       _path_info,
15.       v33,
16.       v34,
17.       *dword_33528);
18.     printf("Location:%s\r\n\r\n", url_buffer);
19.     goto LABEL_58;
20.   }
```

The `sprintf` at line 7 uses a dynamically allocated buffer (`url _ buff`) of 4096 bytes, and copies `path _ info` into it without bounds checking. If `path _ info` exceeds the allocated buffer size, this can lead to a heap overflow.

In our testing, the `ssi` program crashed when attempting to free `file _ buff`, after heap chunk metadata was overwritten by the overflow (snippet also in the `ssi` main function):

```
21. LABEL_58:
22.     free(form_data);
23.     free(file_buff);
24.     free(url_buff);
25.   }
```

We were unable to turn this into remote code execution on Silex SD-330AC. However, because an attacker can overwrite heap metadata, remote code execution may still be possible on some systems depending on allocator behavior and build configuration.

CVE-2026-32960: Authentication bypass via sx_smpd credential reuse

A failure to clear TCP data buffers in the `sx _ smpd` protocol can allow an attacker to reuse stored administrator credentials by sending a spoofed authentication packet. This incomplete cleanup of sensitive data can enable authentication bypass via this protocol.

When processing AMC manager traffic, the `sx _ smpd` daemon checks for an authentication packet containing the administrator password:

```
// Listing 1
01: int __fastcall smpd_prtcl_tcp_get_devaddr(struct config_blob *blob, int fd, _DWORD *a3) {
02:     \\...
03:     node = smpd_find_node(blob, &blob->rcv_buffer[8]);
04:     if ( node || (node = smpd_get_node(blob, &blob->rcv_buffer[8])) != 0 )
05:     {
06:       memcpy(node->header_buff, rcv_buffer, 4 * (blob->rcv_buffer[4] & 0xF));
07:       _recv_len = bswap32(*&blob->rcv_buffer[16]);
08:       node->recv_len = _recv_len;
09:       if ( _recv_len )
10:       {
11:         if ( smpd_alloc_node_buffer(blob, node, _recv_len)
12:           || recv(fd, node->data_buff, node->recv_len, 256) < node->recv_len )
13:         {
14:           return -1;
15:         }
16:         // ...
17:       }
18:       // ...
19:     }
20:   // ...
21: }
```

If a specific AMC manager instance is “known” to the device, `sx _ smpd` looks up a corresponding record (node) using its MAC address (line 3, `smpd _ find _ node()`). If not found, it allocates a new record (line 4, `smpd _ get _ node()`). It then parses the packet header and receives the payload which is stored in heap memory (`node->data _ buff`) in cleartext.

```
// Data buffer example
00084B20  00 00 00 00 09 02 00 00  00 00 00 01 00 00 00 00  ................
00084B30  10 3C 01 00 00 00 00 00  00 00 00 13 00 00 00 01  .<..............
00084B40  00 00 00 02 01 01 00 07  77 6F 6C 6F 6C 6F 00 00  ........wololo..
00084B50  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00084B60  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00084B70  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00084B80  00 00 00 00 00 00 00 00  00 00 00 00 79 14 00 00  ............y...
```

Later, `sx _ smpd` compares the password retrieved from `node->data _ buff` with the configured administrator password (stored on the stack as `blob->password`):

```
// Listing 2
01: int __fastcall smpd_conf_auth_config(struct config_blob *blob, int a2, struct node *node, _DWORD *a4)
02: {
03:   const char *node_password;
04:
05:   if ( a2 )
06:   {
07:     // ...
08:     LOG(blob, 1, "%s():data : %s\n", "smpd_conf_auth_config", node->data_buff);
09:     sub_2454C(blob);
10:     node_password = node->data_buff;
11:     if ( !strcmp(blob->password, node_password) )
12:     {
13:       LOG(blob, 2, "%s():password match!\n", "smpd_conf_auth_config");
14:       return 0;
15:     }
16:     LOG(blob, 1, "%s():password not match [%s]:[%s]\n", "smpd_conf_auth_config", blob->password, node_password);
17:     v9 = 770;
18:     // ...
19:   }
20:   // ...
21: }
```

If authentication succeeds, the AMC manager instance can execute privileged commands, such as retrieving device configuration, altering settings, and updating firmware remotely.

We identified multiple issues in this design:

- **Weak identity**: AMC Manager records are keyed only by a MAC address that is provided in the `sx _ smpd` packet header. This value can be spoofed and is not derived from the link layer
- **Sensitive buffer reuse**: Data buffers associated with AMC manager records (including `node->data _ buff`) are not cleared, allowing reuse of previously stored credentials

A representative exploitation flow is:

1. A legitimate AMC manager authenticates and performs actions, resulting in a node record whose `node->data _ buff` contains a valid password
2. An attacker sends a forged authentication packet that includes the MAC address of the legitimate AMC manager, so the existing node record is reused, but omits a payload (only the header is sent)
3. When `sx _ smpd` processes the attacker’s packet, it overwrites only the header portion. The remaining data in `node->data _ buff`, including the previously stored password remains intact
4. Because the password check uses the same buffer address and a straightforward string comparison, the check succeeds, and the attackers becomes authenticated
5. The attacker can then send additional crafted packets to retrieve the full device configuration. If the administrator password is included in the returned configuration, the attacker can recover it and authenticate normally going forward

CVE-2026-32961: Heap overflow in sx_smpd UDP packet processing (unvalidated data length)
The length of the data portion of `sx _ smpd` packets sent over UDP is not validated. This can lead to heap-based buffer overflows and other memory corruption issues.

Below is an example of an `sx _ smpd` UDP header. The final two octets (`de ad` in this example) represent the payload length that follows the header.

```
0000    53 4d 00 01 08 00 00 00 08 00 27 38 3e 75 00 00
0010    00 00 02 28 dc 2d 06 00 01 00 00 00 00 00 00 00
0020    00 00 00 01 00 00 00 00 10 3c 01 00 00 00 00 00
0030    00 00 02 14 00 00 00 01 00 00 00 02 01 01 de ad
```

The following snippet shows how the UDP data payload is copied into an internal buffer:

```
1. // ...
2. if (dword1FC == 1) {
3.     // ...
4.     memcpy(node->data_buff, data, bswap32(blob->data_len));
5.     node->recv_len = bswap32(blob->data_len);
6.     node->recv_data = node->data_buff;
7. }
   // ...
```

The UDP payload and its length (`blob->data _ len`, line 4) come directly from the incoming packet, and there are no checks to ensure `node->data _ buff` can hold it. `node->data _ buff` is a fixed-size 512-byte heap buffer allocated in `smpd _ get _ node()`. If the UDP payload exceeds the size of `node->data _ buff`, `memcpy()` can trigger a heap buffer overflow, leading to denial of service or remote code execution.

Later, the same payload is copied again in `smpd _ cmdset _ parse _ tags()`:

```
// ...
1. v25 = __rev16(*(v19 + 3));
2. *(v21 + 2) = v25;
3. v26 = malloc(v25);
4. *(v21 + 3) = v26;
5. if ( !v26 )
6.   break;
7. LOG(a1, 1, "%s():   config %d.%d.%d  : %d\n", "smpd_cmdset_parse_configs", v22, v23, v24, v25);
8. memcpy(*(v21 + 3), v19 + 8, *(v21 + 2));
// ...
```

In this later copy, the payload length octets are used to allocate a new buffer, and the subsequent `memcpy()` does not cause a heap overflow by itself. However, because the initial copy disregards the length field while the later copy uses it, an attacker may be able to influence heap layout (for example, where the second buffer is allocated relative to the first). This can provide additional leverage for exploitation. Depending on the environment, the outcome could include denial of service or remote code execution.

CVE-2026-32964: Configuration injection via jcpd (Serial Device Server Setup)
When configuring the device via the `jcpd` protocol (also known as “Serial Device Server Setup”), configuration command arguments do not filter certain special characters (for example, newlines). This allows unauthenticated attackers to inject arbitrary entries into system configuration files.

When sending configuration values via `jcpd`, the following function invokes the local `/tmp/spshttpio` socket to retrieve and set system parameters:

```
int __fastcall send_spshttpio(int sockfd, char *cmd, char *params, signed int len)
{
  sprintf(cmd + 8, "%d", len);
  if ( send(sockfd, cmd, 16u, 0) <= 15 )
    return -1;
  if ( len <= 0 )
    return 0;
  if ( len > send(sockfd, params, len, 0) )
    return -1;
  return 0;
}
```

For example, `jcpd` can set the WiFi SSID. Under the hood, the device executes the following command via the `/tmp/spshttpio` socket:

`SXSETV\x00\x00\xDE\xAD\x00\x00\x00\x00\x00\x00setting=value`

In this structure, `SXSETV` sets a specific configuration key and value separated by =, and the octets `\xDE` and `\xAD` reflect the argument length.

On Silex SD-330AC, only a limited set of configuration values can be modified via `jcpd`, including `wifi _ ssid`. Because the function does not filter special characters, such as newlines, an attacker can inject additional configuration entries. For example:

`SXSETV\x00\x00\x32\x32\x00\x00\x00\x00\x00\x00wifi _ ssid=\nhello=world`

This results in an injected `hello=world` entry in `/etc/wlan/hostapd.conf`, as shown below:

```
#
# This file was automatically generated. Do not edit!
#
interface=wlan0
bridge=br0
dump _ file=/tmp/hostapd.dump
ssid=
hello=world
channel=11
# ....
```

On Silex SD-330AC this can enable injection of arbitrary configuration into `/etc/wlan/hostapd.conf`, altering WiFi security configuration, creating rogue SSIDs, and causing other impacts.

On other products that implement `jcpd` similar injection might also enale OS command injection depending on how the system scripts process the affected configuration files.

Lantronix Vulnerability Details

CVE-2025-67039: Authentication bypass on management pages
Authentication on many management pages can be bypassed by appending a suffix such as `.gif` to the URL and sending an `Authorization` header with the username set to admin. For example, an attacker can send an HTTP Basic header of the form
`Authorization: Basic [base64("admin:password")](base64(%22admin:password%22))).`

The root cause is an Nginx configuration rule that exempts requests for paths ending in certain file extensions (for example, `.gif, .jpg`) from authentication:

```
location ~* \.(gif|jpg|jpeg|png|ico|json)$ {
root /http/config/;
proxy_set_header X-Real-IP  $remote_addr;
proxy_set_header X-Forwarded-For $remote_addr;
proxy_set_header Host $host:80;
proxy_pass http://127.0.0.1:8080;
auth_basic off;
auth_digest off;
}
```

Because this exemption can apply to management paths, a POST request to `/.gif` can bypass Nginx authentication.

For some pages, the backend service (`ltrx _ evo`) performs a strict check on the requested path and rejects requests ending with an arbitrary suffix. For other pages, the validation is weaker and only checks a prefix of the path, enabling bypasses (for example, downloading configuration without authentication by appending `.gif`).

Example of a strict check in `ltrx _ evo`:

```
if ( !strcmp(req_path, "/reboot") )
    {
       (sub_A05A8)(v4);
            //...
    }
```

Example of a weak check that enables unauthenticated configuration download via suffix bypass:

```
if ( !strncmp(req_path, "/export/config", 14) )
    goto LABEL_37;
    //...
```

In addition, some management pages validate user identity using only the username supplied in the `Authorization` header (for both Basic and Digest authentication). As a result, sending an `Authorization` header with `admin` as the username —even with invalid credentials—can be sufficient to bypass this additional check.

CVE-2025-67041: OS command injection in TFTP client (Filesystem Browser)
The `host` parameter of the TFTP client in the Filesystem Browser page is not properly sanitized. In particular, the sanitization routine does not block single quotes and newline characters. These characters can be used to break out of the intended command context and execute arbitrary commands as root.

The handler constructs a command for TFTP operations similar to the following:

```
v58 = 0;
v47 = sprintf_malloc("tftp -l '%s/%s' -r '%s' -p '%s' %d 2>&1", "/ltrx_user", dest, remote, host, port);
v57 = 0;
(exec_system_cmd_ex)(v47, &v57, &v58);
```

The `host` parameter is retrieved and sanitized in the following way:

```
v24 = get_value(a1, "host", v22);
host = v24;
//[...]
v25 = strlen(v24);
v27 = host;
v28 = &host[v25];
while ( v28 != v27 )
{
  v41 = *v27++;
  v42 = (v41 - 33);
  v43 = v41 == '`';
  v44 = v41 & 0xDF;
  if ( v42 > 0x1D )
    LOBYTE(v45) = 1;
  else
    v45 = ~(0x2C000029u >> v42);
  if ( v44 == '\\' )
    v26 = v43 | 1;
  else
    v26 = v43;
  if ( v26 | !(v45 & 1) )
  {
    v7 = 's';
    goto LABEL_12;
  }
}
```

This function iterates over the ASCII characters in `host`, computes each character's offset from `!` (ASCII 33) and checks whether it falls within the printable range up to `>` (ASCII 62). It then uses a 32-bit bitmask (`0x2C000029u >> v42`) to identify disallowed characters and rejects any input for which the corresponding bit is set. As a result, the routine blocks specific symbols (`!, $, &, ;, <, >`) as well as backticks and backslashes but it does not block single quotes or newline characters, which can be used to concatenate multiple commands. This vulnerability can be chained with CVE-2025-67039 to reach RCE without authentication.

CVE-2025-67034: Authenticated OS command injection in SSL credentials
The management interface allows users to add and delete SSL credentials. During deletion, the name parameter is not sanitized and is concatenated into a command that is executed with `luci.sys.exec`, allowing authenticated command injection as root.

Vulnerable code path (`ltrx-ssl.lua`, line 29):

```
function deletecredential()
    local mArray={}
    local cert_type = luci.http.formvalue("authority")
    local credentialName = luci.http.formvalue("name")
    /*[...]*/
    if cert_type == "credential" then
        mArray[0]=luci.sys.exec('ltrx_mhcfgupdate "delete ssl credential" ' .. credentialName .. '')
    end
    if cert_type == "trusted" then
        mArray[0]=luci.sys.exec('ltrx_mhcfgupdate "delete trusted authority" ' .. credentialName .. '')
    end
    if cert_type == "intermediate" then
         mArray[0]=luci.sys.exec('ltrx_mhcfgupdate "delete intermediate authority" ' .. credentialName .. '')
    end
    luci.util.perror("delete " .. credentialName .. " credential" .. cert_type .. mArray[0])
    luci.http.prepare_content("application/json")
    luci.http.write_json(mArray)
end
```

The `name` parameter is retrieved from the HTTP request and stored in `credentialName`. The code concatenates `credentialName` into a shell command and executes it in one of the if branches.

CVE-2025-67038: Unauthenticated OS command injection via RPC authentication logging
The HTTP RPC module writes a log entry after failed authentication attempts by executing a shell command. The username is directly concatenated into the log string without input sanitization, and the log string is then executed using `os.execute`. This can allow command injection as root

Vulnerable code path (`rpc.lua`, line 103):

```
local msg = "User '" .. user .. "' failed to login from RPC,"
local str = timestamp .. "," .. user .. "," .. (http.getenv["REMOTE_ADDR"] or "?") .. ",User,Login," .. msg
str = str .. "Failed"
os.execute("echo -n \"" .. str .. "\" > " .. tmpname)
os.execute("mv " .. tmpname .. " /var/auth/")
```

The username is directly concatenated into the command string. An attacker can supply a crafted username containing shell metacharacters to inject arbitrary commands executed with root privileges.

CVE-2025-67036: Authenticated OS command injection in Log Info
The Log Info page allows users to view log files by specifying a filename. Due to missing sanitization of the file name parameter, an authenticated attacker can inject OS commands that are executed as root.

Vulnerable code path (`action _ upload` in `log _ info.lua`, line 194):

```
194 if file then
195     local filetosend
196     if string.sub(name, -1) == "/" then
197         filetosend = name..file
198     else
199         filetosend = name.."/"..file
200     end
201     if string.find(server,"local") then
202         local cmd = "cat "..filetosend.." 2>/dev/null"
203         local reader = ltn12_popen(cmd)
```

The code takes file from user input via `luci.http.formvalue("filename")` and a GET request. It variable concatenates this value into a command string (`cmd`) without sanitization and executes it via `/bin/sh -c` through `ltn12 _ popen` (via `nixio.exec`). An attacker can supply a crafted filename containing shell metacharacters (for example, `foo; rm -rf /; #`) to execute arbitrary commands as root.

CVE-2025-67035: Multiple authenticated OS command injections In SSH pages
The SSH Client and SSH Server pages include multiple command injection issues due to missing sanitization of input parameters. An attacker can inject arbitrary commands into delete actions for various objects, such as server keys, users, and known hosts. Injected commands are executed with root privileges.

In these delete actions, the form parameters concatenated into shell commands and executed by `luci.sys.exec`.

Vulnerable code path (`ltrx-ssh.lua`):

```
function deleteserverkeys()
local mArray={}
    local action = luci.http.formvalue("action")
    luci.util.perror("delete serverkeys action: " .. action)
    local keytype = luci.http.formvalue("type")
    luci.util.perror("delete server keytype: " .. keytype)
    if action == "deleteserverkey" then
       mArray[0]=luci.sys.exec('ltrx_mhcfgupdate "deleteserverkeys" '.. keytype ..'')
    end
    luci.http.prepare_content("application/json")
    luci.http.write_json(mArray)
end
```

`keytype` is retrieved from the HTTP request through `luci.http.formvalue("type")`, concatenated into a command string and executed. The same issue affects multiple functions in `ltrx-ssh.lua` including `deleteauthorizeduser()`, `deleteclientknownhosts()`, and `deleteuser()`

There may be additional variants of this pattern in other pages.

CVE-2025-67037: Authenticated OS command injection in tunnel termination (Serial Page)
An authenticated attacker can inject OS commands into the `tunnel` parameter when killing a tunnel connection. Injected commands executed as root.

Vulnerable code path (`serial.lua`, line 429):

```
function killconnection()
    local mArray={}
    local oneBasedIndex = luci.http.formvalue("tunnel")
    local mode = luci.http.formvalue("mode")
    --mArray[0]=luci.sys.exec("ls")
    if mode == "accept" then
            luci.sys.exec("/usr/sbin/tunnelkill.sh \"tunnel " .. oneBasedIndex .. "\" accept \"kill connection\"")
    end
    if mode == "connect" then
            luci.sys.exec("/usr/sbin/tunnelkill.sh \"tunnel " .. oneBasedIndex .. "\" connect \"kill connection\"")
    end
    luci.util.perror("kill " .. mode .. " connection " .. oneBasedIndex)
    luci.http.prepare_content("application/json")
    luci.http.write_json(mArray)
end
```

The tunnel parameter is retrieved from the HTTP request, stored in `oneBasedIndex`, concatenated into a command and executed via `luci.sys.exec`.